

MacOS Endpoint Security Framework: Not Another MacOS Log Source!

By Jacob Latonis & Julia Paluch

Who we are



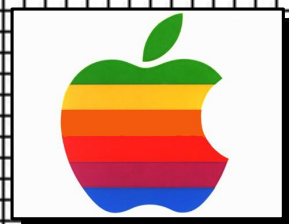
Jacob Latonis



Julia Paluch

both massive nerds

**Let's set some baseline assumptions.
We are focusing on scenarios where:**



There is an ongoing incident.

Events to be investigated happened in the last few days.

You want an understanding of lower-level system events, e.g., entitlement grants, file opens, and fork/exec calls.

You are performing a threat hunt.

OR

**Why are we talking
about this now?**



Why are we talking about this now?

 BleepingComputer

NimDoor crypto-theft macOS malware revives itself when killed

 SentinelOne

macOS.ZuRu Resurfaces | Modified Khepri C2 Hides Inside Doctored Termius App

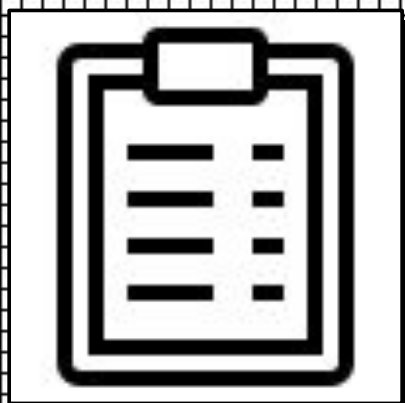
 Cybernews

Russia-linked macOS malware just got a whole lot more dangerous

**What tools and
processes are used in
a typical MacOS
investigation
scenario?**

Typical MacOS Investigations

Sysdiagnose



A diagnostics tool originally meant to be used by MacOS developers for investigating bug reports.

Includes information about running processes, installed applications, and system events.

Can be collected with

```
> sudo sysdiagnose
```

Typical MacOS Investigations

WHAT sysdiagnose COLLECTS:

- A spindump of the system
- Several seconds of top output
- Data about kernel zones
- Status of loaded kernel extensions
- Resident memory usage of user processes
- Recent system logs
- A System Profiler report
- Recent crash reports
- Disk usage information
- I/O Kit registry information
- Network status
- If a specific process is supplied as an argument, will collect:
 - A list of malloc-allocated buffers in the process's heap
 - Data about unreferenced malloc buffers in the process's memory
 - Data about the virtual memory regions allocated in the process

Typical MacOS Investigations

log



A built-in command line tool for managing and investigating system logs (a subset of which can be collected with sysdiagnose).

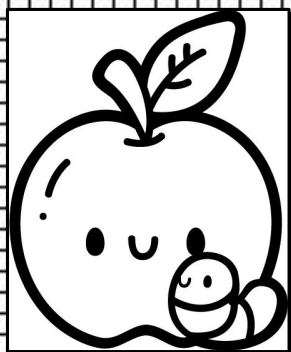
Helpful commands:

- > `log show`
- > `log collect`
- > `log stats`

Investigations are made more powerful with the use of predicate-based filtering.

Typical MacOS Investigations

Apple Unified Log



The system log viewable in the Console App and queryable with the log command.

Contains information about events associated with:

- system logons
- sudo commands
- DNS resolutions
- Gatekeeper events
- Xprotect events
- Apple Script activity
- and much, much, more!

Typical MacOS Investigations

fsevents



.fsevents

A set of files populated by the fsevents process, which monitors file system changes.

Provides filenames associated with file creation, removal, move, permission change, etc.

Remember!

No timestamps :(

Updating the OS will cause the contents of the fsevents folder to be wiped.

Some file system events may not be recorded.

Enter Endpoint Security Framework



System Extension Blocked

A program tried to load new system extension(s) signed by "Legacy Developer: Hp". You can enable these extensions in Privacy & Security System Settings.

OK

Open System Settings

An API released in 2019 that provides visibility into lower-level system events, such as file opening, process execution, and system calls.

Starting with MacOS 15.4, modifications to the TCC database will trigger Endpoint Security events.

Some
notable
events:

exec
fork
create
open
tcc_modify
unlink

Notable ES Events

exec

indicates the
execution of a
process

open

indicates the
opening of a file

create

indicates the
creation of a file

unlink

indicates the
deletion of a file

Notable ES Events

utimes

indicates a change to a file's access time or modification time

tcc_modify

indicates a modification to the TCC database

uipc_connect

indicates the connection of a socket

btm_launch_item_add

indicates the creation of a launch item

Notable ES Properties

es_process_t struct

✓ [S] es_process_t

Inspecting the Source Process

[P] var audit_token: audit_token_t

[P] var executable: UnsafeMutablePointer<String>

[P] var is_es_client: Bool

[P] var is_platform_binary: Bool

[P] var start_time: timeval

pointer to a struct
containing executable
file path and metadata

indicates whether the
binary is signed with
Apple signatures

represents the time
that a fork call
created the process

Notable ES Properties

es_process_t struct

Inspecting Process IDs

```
(P) var ppid: pid_t  
(P) var original_ppid: pid_t  
(P) var group_id: pid_t  
(P) var session_id: pid_t  
(P) var tty: UnsafeMutablePointer<es_file_t>?
```

Inspecting Code Signing Properties

```
(P) var codesigning_flags: UInt32  
(P) var cdhash: es_cdhash_t  
(P) var signing_id: es_string_token_t  
(P) var team_id: es_string_token_t
```

Inspecting Audit Tokens

```
(P) var responsible_audit_token: audit_token_t  
(P) var parent_audit_token: audit_token_t
```

PID of the parent process

PID of the original parent process

ID of the process group

ID of the process session

token that can be used to extract information about the process

How do I collect these events?

eslogger

Example

```
sudo eslogger  
exec fork exit >>  
/tmp/proc.log
```

will subscribe to
exec, fork, and exit
(process lifecycle
events) and append
them to a file

A built-in tool for logging ES events to
stdout in JSONL format or to the unified
logging system.

Must be run with sudo and requires responsible
process to have Full Disk Access.

--list-events

lists supported events

--oslog

emit events to the unified logging system

****Does not support auth events, and
some fields are unavailable.****

create & exec examples

```
void create_launch_agent() {  
    FILE *fp = fopen(LAUNCH_AGENT_PLIST_PATH, "w");  
    if (fp) {  
        fprintf(fp,  
            "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n"  
            "<!DOCTYPE plist PUBLIC \"-//Apple/DTD PLIST 1.0//EN\" \"  
            \"http://www.apple.com/DTDs/PropertyList-1.0.dtd\">\n"  
            "<plist version=\"1.0\">\n"  
            "<dict>\n"  
            "    <key>Label</key>\n"  
            "    <string>%s</string>\n"  
            "    <key>ProgramArguments</key>\n"  
            "    <array>\n"  
            "        <string>%s</string>\n"  
            "    </array>\n"  
            "    <key>RunAtLoad</key>\n"  
            "    <true/>\n"  
            "</dict>\n"  
            "</plist>\n"  
            LAUNCH_AGENT_LABEL, PAYLOAD_PATH);  
        fclose(fp);  
        char cmd[512];  
        sprintf(cmd, sizeof(cmd), "launchctl load %s", LAUNCH_AGENT_PLIST_PATH);  
        system(cmd);  
    }  
}
```

Create Details: {
 "time": "2025-07-16T02:38:14.023319215Z",
 "pid": 770,
 "ppid": 752,
 "rpid": 750,
 "executable_path": "/Users/user/oopsie",
 "file_path": "/Users/Shared/com.malicious.persistence.plist"
}

Exec Details: {
 "time": "2025-07-16T02:38:14.027979798Z",
 "pid": 772,
 "ppid": 770,
 "rpid": 750,
 "executable_path": "/bin/launchctl",
 "args": [
 "launchctl",
 "load",
 "/Users/Shared/com.malicious.persistence.plist"
]
}

unlink event example

```
#include <limits.h>
#include <mach-o/dyld.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    char path[PATH_MAX];
    uint32_t size = sizeof(path);

    if (_NSGetExecutablePath(path, &size) != 0) {
        fprintf(stderr, "Failed to get executable path\n");
        return 1;
    }

    printf("Running program. Will delete: %s\n", path);

    if (unlink(path) != 0) {
        perror("Failed to delete self");
        return 1;
    }

    printf("Deleted self successfully.\n");
    return 0;
}
```

Unlink Details: {

- "time": "2025-07-16T04:24:07.906249213Z",
- "pid": 863,
- "ppid": 851,
- "rpid": 711,
- "executable_path": "/Users/user/delete_me",
- "file_path": "/Users/user/delete_me"

}

open event example

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main() {
    const char *home = getenv("HOME");
    if (!home) {
        fprintf(stderr, "Could not get HOME environment variable.\n");
        return 1;
    }

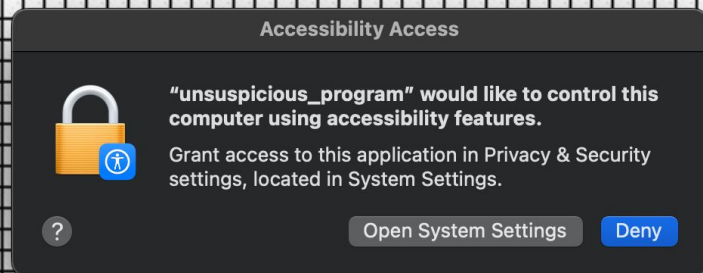
    char path[4096];
    snprintf(path, sizeof(path), "%s/.wallet", home);

    FILE *file = fopen(path, "r");
    if (!file) {
        perror("Failed to open .wallet file");
        return 1;
    }

    printf("Successfully opened %s\n", path);
    fclose(file);
    return 0;
}
```

Open Details: {
 "time": "2025-07-08T14:27:35.200795350Z",
 "pid": 1937,
 "ppid": 1932,
 "rpid": 410,
 "executable_path": "/Users/user/Downloads/chrome_update",
 "file_path": "/Users/user/.wallet"
}

TCC DB



Transparency, Consent, and Control

The `tcc_modify` event indicates that a modification was made to the TCC database.

How does it work?

When an application attempts to access a protected resource, the request is routed through the `tccd` process.

First time executing a program?

yes: Prompt the user for permission

no: Check TCC database for permission



An example `tcc_modify` event

```
{
  "time": "2025-07-08T16:52:10.496460681Z",
  "instigator": "/System/Library/ExtensionKit/Extensions/SecurityPrivacyExtension.appex/Contents/MacOS/SecurityPrivacyExtension",
  "pid": 149,
  "ppid": 1,
  "rpid": 149,
  "executable_path": "/System/Library/PrivateFrameworks/TCC.framework/Support/tccd",
  "service": "Accessibility",
  "right_details": {
    "authorization": "ALLOWED",
    "allowed_identity": "com.apple.Terminal"
  }
}
```

If the **instigator** is the **Security Privacy Extension**, this means that the user performed the action through System Settings.

The **right_details** struct contains the new authorization and what entity it was granted to.

The **service** field indicates the TCC service that was modified.

The **Accessibility** service is one that is needed by apps that help users control their device.

Other examples include **Camera**, **Microphone**, and **Screen Capture**.

A little more about the TCC DB...



The TCC database is technically owned and readable/writeable by the current user.

Nowadays, directly modifying the database is protected by System Integrity Protection (SIP), so even if a program is running as root, it cannot modify the database unless it has specific entitlements.

But back in the day, you could technically write to the database yourself...



SANDER VAN DRAGT

SEPTEMBER 22, 2016

Dropbox Modifies TCC.db to Give Itself Accessibility Access

How can you leverage ES in an investigation?

Incorporate eslogger into investigatory tasks, such as baselining a system and listening for anomalous activity.

A simple Python script or even jq in the command line is enough to pull out the information you need.

Ensure that any EDRs available to you are configured to listen to ES events, if possible.



Though the C API is the preferred and most complete way to listen for ES events, it is not feasible for simple programs because binaries that call the ES API must be notarized by Apple.



Want to learn more?

Blue Teaming on macOS with eslogger

Cybereason

Endpoint Security in a macOS World

Huntress

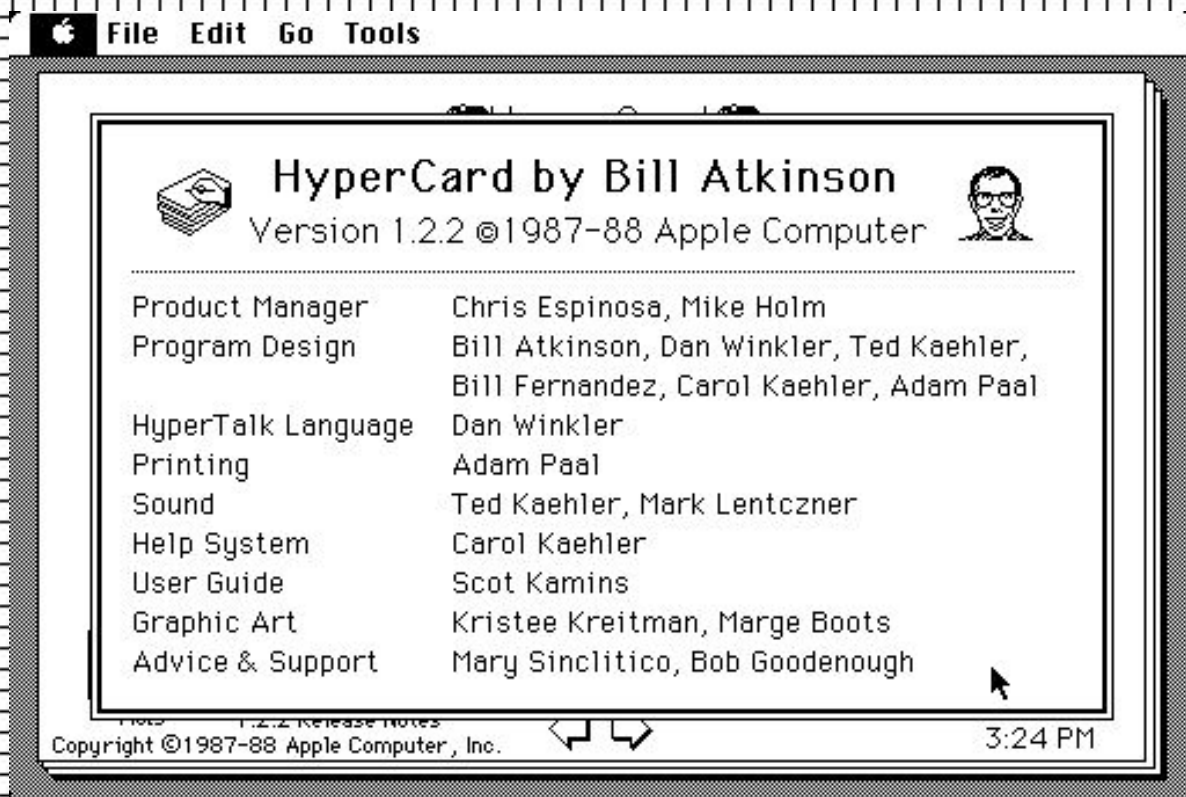
TCCing is Believing

Objective-See

Threat Hunting Pids Within Apple's Endpoint Security API

The Mitten Mac

Why do your slides look like that?





Questions?

pls no hard ones